

Laporan Plea of the Sinful Spirit



Anindya Naufal Pinasthika

13524013

Daftar isi

Daftar isi.....	2
Lingkungan VM dan Verifikasi Dukungan eBPF.....	3
1. Overview Komponen dan Arsitektur Solusi.....	4
a. User-Space Loader.....	4
b. Program eBPF.....	5
c. Komunikasi Antar Komponen.....	5
2. Pilihan Tooling dan Alasan Pemilihan.....	6
3. Execution Flow End-to-End.....	7
1. main() entripoint.....	7
2. Load Program BPF.....	7
3. Mengirim Konfigurasi BPF.....	8
4. Pemasangan Hook.....	9
5. eBPF Activity.....	9
6. Ring Buffer.....	10
7. Loader Menerima Event/Statisik.....	10
Struktur Data dan BPF Map.....	12
a. Map Statistik Per-CPU.....	12
b. Runtime Configuration.....	13
c. Ring Buffer.....	14
d. Inflight Syscall.....	14
e. State Rate Limiting.....	15
Hook Point yang Dipilih dan Alasannya.....	17
Implementasi Bonus.....	18
a. Page Fault Hook.....	18
b. CPU-Level Rate Limiting.....	18
c. Pembuktian Tidak Ada Data Race.....	20
d. Fentry/Fexit.....	20
Hasil Verifier dan Benchmark.....	22
a. Verifier.....	22
b. Benchmark.....	22

Lingkungan VM dan Verifikasi Dukungan eBPF

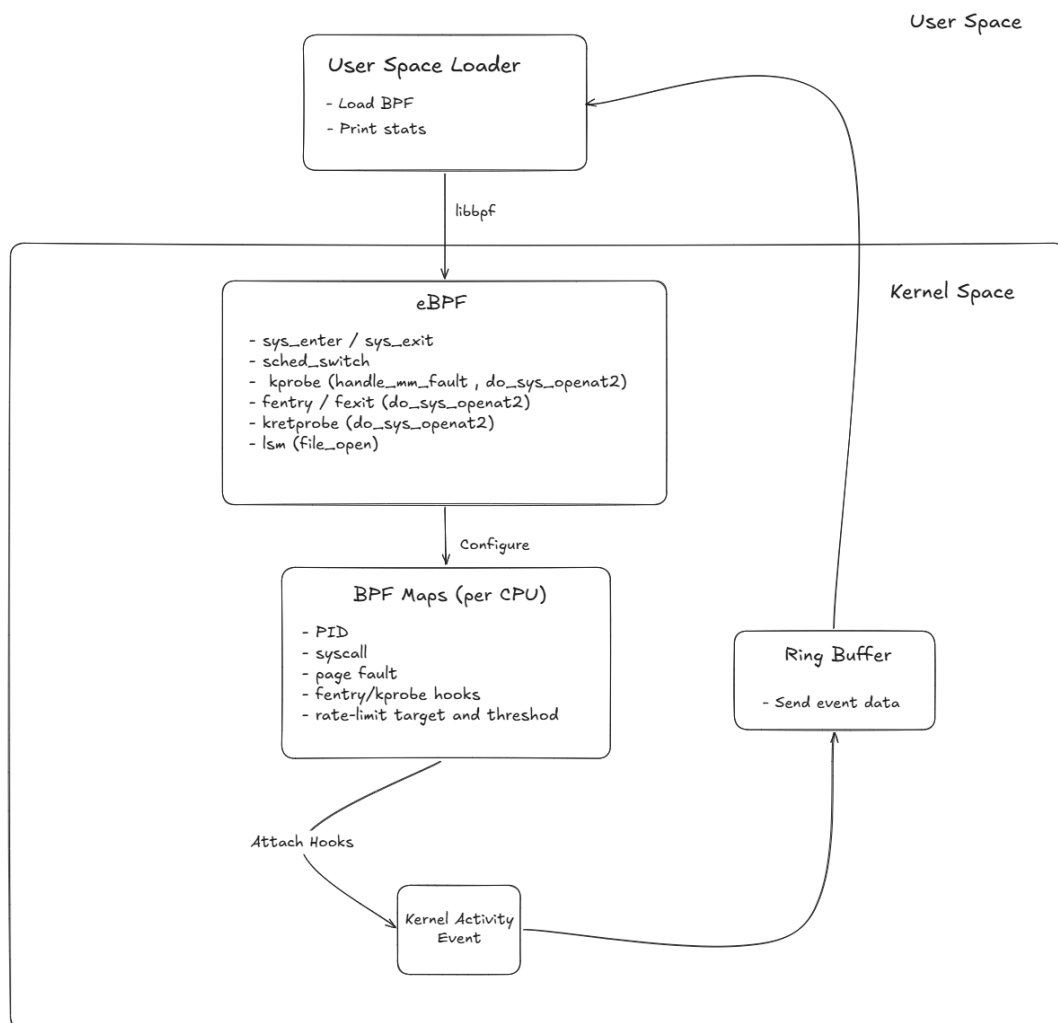
Item	Verifikasi	Hasil
Versi kernel	<code>uname -r</code>	6.8.0-137-generic
BTF vmlinux	<code>test -r /sys/kernel/btf/vmlinux && echo BTF_OK</code>	BTF_OK
Feature probe	<code>sudo bpftool feature probe kernel</code>	(ter-load dengan baik, hasil aktual terlalu besar untuk ditaruh di sini)
Dukungan kernel BPF LSM	<code>grep CONFIG_BPF_LSM /boot/config-\$(uname -r)</code>	CONFIG_BPF_LSM=y
Konfigurasi boot BPF LSM	<code>grep -o 'lsm=[^]*' /proc/cmdline</code>	lsm=lockdown,capability,landlock,yama,apparmor,bpf
BPF LSM aktif	<code>cat /sys/kernel/security/lsm</code>	lockdown,capability,landlock,yama,apparmor,bpf

1. Overview Komponen dan Arsitektur Solusi

Program dibagi menjadi dua lapisan utama:

- eBPF di kernel
- Loader/collector di user space

Jalur kernel menangani observasi event dan update state yang harus dekat dengan sumber event. User space menangani konfigurasi, lifecycle program, konsumsi event, perhitungan rate, dan formatting output. Berikut arsitektur gambaran kasar bagaimana program eBPF bekerja



Pada program eBPF terdapat beberapa lapisan yang bekerja:

a. User-Space Loader

Bertugas membaca file biner eBPF, menyiapkan objek seperti *maps*, lalu memuat instruksi tersebut ke dalam kernel Linux menggunakan system call `bpf()` (yang diabstraksikan dalam operasi `libbpf`). Kurang lebih loader bertanggung jawab untuk:

- Memeriksa dukungan kernel dan BTF.
- Mengatur konfigurasi runtime melalui BPF map.
- Membaca event dari ring buffer.
- Membaca statistik dari per-CPU BPF map.
- Menampilkan log event dan statistik live.

b. Program eBPF

Merupakan suatu program .bpf yang diimplementasikan dengan bantuan struktur `vmlinux.h` dari `/sys/kernel/btf/vmlinux`. Awalnya .bpf ini merupakan suatu bytecode hasil compile program. Namun, setelah program di ubah dengan JIT (just in time compile) oleh `libbpf` dengan penyesuaian struct (agar mendukung compile once, run everywhere), eBPF akan diubah menjadi binary yang nantinya dapat dimasukkan ke dalam kernel space. Di mana binary eBPF ini harus memenuhi beberapa aturan strict yang diberikan kernel dalam bentuk verifier (tidak boleh infinite loop, mengakses pointer/memory yang dilarang, dsb). Program eBPF sendiri bertanggung jawab untuk:

- Mengambil informasi dari aktivitas kernel.
- Mencatat metadata seperti PID, nama proses, CPU core, dan timestamp.
- Memperbarui statistik pada per-CPU BPF map.
- Mengirim data event ke user space melalui ring buffer.

c. Komunikasi Antar Komponen

Secara kasar, loader dan eBPF melakukan komunikasi back and forth melalui BPF maps serta ring buffer. Di mana BPF map digunakan untuk konfigurasi dan penyimpanan/pencatatan statistik melalui shared memory, sedangkan ring buffer digunakan untuk mengirim event dari kernel space ke user space menggunakan mekanisme polling (eBPF push melalui `bpf_ringbuf_submit()` , loader menerima melalui `ring_buffer__poll()`).

Secara umum, loader memuat dan mengonfigurasi program eBPF. Setelah program berjalan di kernel, data yang dikumpulkan dikirim kembali ke loader untuk ditampilkan sebagai log per-event dan statistik live per CPU.

2. Pilihan Tooling dan Alasan Pemilihan

Tool / Teknologi	Alasan
C	Bahasa pemrograman yang works best dalam implementasi system related implementation (apalagi yang berhubungan dengan kernel dalam konteks ini). Selain itu, C dapat memberikan kontrol yang eksplisit, layout struct, syscall, dan integrasi native dengan libbpf.
Clang target bpf	Mengompilasi program bpf menjadi bytecode eBPF dengan -target bpf.
libbpf	Menyediakan abstraksi fungsi yang sudah mengelola object BPF, map, link, ring buffer, BTF, serta lifecycle load/attach dengan API yang sudah disediakan.
bpftool	Membuat vmlinux.h, menghasilkan skeleton, melakukan feature probe, serta membuktikan program dan map benar-benar ter-load.
Makefile	Mengompile seluruh proyek eBPF, termasuk pembuatan vmlinux.h, BPF object, skeleton, executable loader, dan tools-tools lainnya.

3. Execution Flow End-to-End

1. main() entryptoint

Bekerja dengan meload tiap struktur data yang saat ini nanti dibutuhkan dan melakukan parsing argumen opsional yang saat ini ada sebelum nantinya dilanjutkan untuk me-load libbpf nya

```
int main(int argc, char **argv)
{
    struct cpuwatch_cpu_stats *previous = NULL, *current = NULL;
    struct cpuwatch_cpu_rate *rates = NULL;
    struct cpuwatch_bpf *skel = NULL;
    struct ring_buffer *ring = NULL;
    struct app_context app = {0};
    struct app_options opts;
    struct timespec started, sampled, now;
    struct utsname uts = {0};
    bool terminal_sample_done = false;
    int nr_cpus, stats_fd, error = 0;
    ...
}
```

2. Load Program BPF

Dilakukan dengan awalnya menginisialisasi libbpf dan skeleton ebpf. Fungsi dari ini adalah program dapat mempunyai beberapa API sehingga kita bisa mengakses eBPF, BPF map, dsb lewat skeleton (wrapper hasil generate bpftool). Setelah itu, program akan di-load di kernel (di mana eBPFnya akan dilalui beberapa proses verifikasi oleh verifiator kernel yang telah disebutkan sebelumnya, untuk memastikan apakah program eBPF aman untuk di-load ke kernel)

```
// init libbpf (strict mode), then init skeleton ebpf
// skeleton ebpf is kinda like boilerplate so that itll easy to use
libbpf_set_strict_mode(LIBBPF_STRICT_ALL);
skel = cpuwatch_bpf__open();
if (!skel) {
    fprintf(...);
    return EXIT_FAILURE;
}

// check if the loaded hook exist/supported
```

```

    if (configure_autoload(skel, &opts)) {
        error = -1;
        goto cleanup;
    }

    // check if ebpf is successfully loaded & verified in the kernel
    if (cpuwatch_bpf__load(skel)) {
        fprintf(...);
        error = -1;
        goto cleanup;
    }

```

Selain verifier, program juga akan mengecek apakah hook yang disiapkan benar-benar available atau tidak pada `configure_autoload` untuk memverifikasi hooknya tersedia atau tidak.

3. Mengirim Konfigurasi BPF

Proses mentransfer konfigurasi merupakan suatu proses transfer data parameter, aturan, atau status dari loader ke dalam kode eBPF nantinya. Karena kode kernel eBPF tidak bisa membaca file secara langsung, konfigurasi ini biasanya dikirim menggunakan struktur data eBPF Maps. Proses dilakukan dalam fungsi `update_config`

```

// send configuration to kernel, it will be send to map afterward
if (update_config(skel, &opts)) {
    error = -1;
    goto cleanup;
}

```

Di mana isi dari fungsi `update_config` sebagai berikut

```

static int update_config(struct cpuwatch_bpf *skel, const struct app_options *opts)
{
    struct cpuwatch_config cfg = {
        .emit_events = opts->events,
        .filter_flags = (opts->have_pid ? CPUWATCH_FILTER_TGID : 0) |
(opts->have_syscall ? CPUWATCH_FILTER_SYSCALL : 0),
        .monitor_tgid = opts->pid,
        .monitor_syscall = opts->syscall_nr,
        .pagefault_enabled = opts->pagefault,
        .fentry_enabled = opts->fentry,
        .kprobe_open_enabled = opts->kprobe_open,
    };
}

```



```

        .rate_limit_enabled = opts->have_rate_limit,
        .rate_limit_tgid = opts->rate_pid,
        .rate_limit_threshold = opts->rate_limit,
        .rate_limit_syscall = SYS_openat,
};
__u32 key = CPUWATCH_CONFIG_KEY;
int fd = bpf_map__fd(skel->maps.runtime_cfg);

if (bpf_map_update_elem(fd, &key, &cfg, BPF_ANY)) {
    fprintf(...);
    return -errno;
}
return 0;
}

```

Di mana konfigurasi intinya dibuat dalam struct eBPFs map `cpuwatch_config`, dan isi dari konfigurasi tersebut disimpan dalam `bpf_map_update_elem()`. Di mana eBPF nanti akan membaca konfigurasi tersebut melalui fungsi `*get_config(void)` (yang di mana akan memanggil `(bpf_map_lookup_elem)`

4. Pemasangan Hook

Hook dipasang ke kernel melalui

```

// attach hook to kernel
if (attach_programs(skel, &opts)) {
    error = -1;
    goto cleanup;
}

```

Di mana `attach_programs` merupakan fungsi abstraksi dari implementasi fungsi `attach hook eBPF bpf_program__attach()`, hook yang diattach sendiri salah satunya:

- `sys.enter/sys.exit`
- `sched switch`
- `kprobe (handle mm fault, do_sys_openat2)`
- `fentry/fexit (do_sys_openat2)`
- `kretprobe (do_sys_openat2)`
- `lsim (file_open)`

Setelah terpasang, kode eBPF mulai dijalankan setiap kali aktivitas tersebut terjadi di kernel.

5. eBPF Activity

Ketika ada suatu event terjadi (misal `syscall`), BPF awalnya mendeteksi mencatatnya lewat

```
SEC("tracepoint/raw_syscalls/sys_enter")
int handle_sys_enter(struct trace_event_raw_sys_enter *ctx)
```

Di mana eBPF nantinya akan mencatat statistik dengan menambah counter & membuat event detail dalam bentuk

```
struct cpuwatch_event_header {
    __u64 timestamp_ns;
    __u32 tgid;
    __u32 pid;
    __u32 cpu;
    __u32 type;
    char comm[CPUWATCH_TASK_COMM_LEN];
};
```

6. Ring Buffer

Ketika eBPF sudah mencatat semua data yang dibutuhkan ketika melewati suatu event kernel via hook (seperti syscall), eBPF akan mengirimnya balik ke loader di user space dengan cara ring buffer. Prosesnya terjadi ketika event telah ter-trigger, eBPF akan menyimpan sementara menggunakan

```
event = bpf_ringbuf_reserve(&events, sizeof(*event), 0);
```

Setelah beberapa proses validasi / update pencatatan statistik, eBPF lalu mengirim dengan cara

```
bpf_ringbuf_submit(event, 0);
```

7. Loader Menerima Event/Statistik

Loader pada userspace membaca event dan statistik menggunakan `ring_buffer__poll` untuk menunggu hingga ada suatu ringbuffer notify dari eBPF terjadi. Proses ini terjadi dalam suatu while loop infinite hingga program di userspace stop.

```
// read the stats per CPU
while (!exiting) {
    error = ring_buffer__poll(ring, 100);
    if (error == -EINTR) {
        error = 0;
    }
}
```

```

        if (error < 0) {
            fprintf(...);
            break;
        }
        clock_gettime(CLOCK_MONOTONIC, &now);
        bool interval_due = elapsed_seconds(&sampld, &now) * 1000.0 >=
opts.interval_ms;
        bool duration_due = opts.duration_sec > 0.0 && elapsed_seconds(&started,
&now) >= opts.duration_sec;
        bool terminal_due = exiting || duration_due;

        if ((interval_due || terminal_due) &&
            sample_and_emit_stats(stats_fd, previous, current, rates, nr_cpus,
&started, &sampld, &now, &opts, uts.release)) {
            error = -1;
            break;
        }
        if (terminal_due) {
            terminal_sample_done = true;
            break;
        }
    }
}

```

Jikalau loader menerima statistik, proses tersebut terjadi pada eBPF maps di mana karena eBPF dan loader menggunakan shared memory, loader bisa langsung membaca data-data statistik menggunakan fungsi `bpf_map_lookup_elem` melalui abstraksi fungsi `read_rates` (yang di mana `read_rates` tersebut dipanggil dari `sample_and_emit_stats()`)

```

static int read_rates(...)
{
    __u32 key = CPUWATCH_STATS_KEY;

    if (bpf_map_lookup_elem(stats_fd, &key, current)) {
        fprintf(...);
        return -errno;
    }
    ...
}

```

Struktur Data dan BPF Map

Berikut beberapa struktur data BPF map dengan fungsi berbeda untuk menyimpan konfigurasi, statistik, state sementara, serta mengirim event dari kernel ke user space.

Map	Tipe	Fungsi
stats	BPF_MAP_TYPE_PERCPU_ARRAY	Menyimpan statistik agregat pada masing-masing CPU.
runtime_cfg	BPF_MAP_TYPE_ARRAY	Menyimpan konfigurasi runtime dari loader.
events	BPF_MAP_TYPE_RINGBUF	Mengirim data event dari kernel ke user space.
inflight	BPF_MAP_TYPE_LRU_HASH	Menyimpan state syscall antara sys_enter dan sys_exit.
target_inflight	BPF_MAP_TYPE_LRU_HASH	Menandai syscall yang menjadi target rate limiting.
rate_states	BPF_MAP_TYPE_PERCPU_HASH	Menyimpan state rate limit per proses pada masing-masing CPU.

a. Map Statistik Per-CPU

Map stats menggunakan BPF_MAP_TYPE_PERCPU_ARRAY, di mana setiap CPU mendapatkan salinan value sendiri.

```
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, struct cpwatch_cpu_stats);
} stats SEC(".maps");
```

Di mana valuenya (cpwatch_cpu_stats) merupakan struct tambahan:

```
// CPU stats
struct cpwatch_cpu_stats {
    __u64 syscall_enter;
    __u64 syscall_exit;
```

```

__u64 context_switches;
__u64 page_faults;
__u64 rate_limited;
__u64 ringbuf_drops;
};

```

Struktur per-CPU memungkinkan setiap CPU memperbarui counter masing-masing sehingga mengurangi lock contention (semacam perebutan akses memori) dibandingkan penggunaan satu counter global yang diakses seluruh CPU.

b. Runtime Configuration

Map runtime_cfg menyimpan konfigurasi global yang diberikan oleh loader kepada program eBPF.

```

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, struct cpuwatch_config);
} runtime_cfg SEC(".maps");

```

Konfigurasi cpuwatch_config sebagai value map menyimpan antara lain filter PID, filter syscall, serta pengaktifan fitur tambahan.

```

// CPU runtime config
struct cpuwatch_config {
    __u32 emit_events;
    __u32 filter_flags;
    __u32 monitor_tgid;
    __s32 monitor_syscall;
    __u32 pagefault_enabled;
    __u32 fentry_enabled;
    __u32 kprobe_open_enabled;
    __u32 rate_limit_enabled;
    __u32 rate_limit_tgid;
    __u32 rate_limit_threshold;
    __s32 rate_limit_syscall;
    __u32 reserved;
};

```

Map ini menggunakan BPF_MAP_TYPE_ARRAY karena hanya diperlukan satu konfigurasi global yang dibaca oleh seluruh CPU.

c. Ring Buffer

Map events menggunakan BPF_MAP_TYPE_RINGBUF sebagai jalur pengiriman event dari kernel ke user space.

```
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 24);
} events SEC(".maps");
```

Di mana BPF_MAP_TYPE_RINGBUF dibungkus dalam max_entries yang dimana implementasinya program eBPF terlebih dahulu melakukan reserve ruang pada ring buffer seperti

```
event = bpf_ringbuf_reserve(&events, sizeof(*event), 0);
```

Setelah data event diisi, record ring buffer baru dikirim dengan

```
bpf_ringbuf_submit(event, 0);
```

d. Inflight Syscall

Menggunakan BPF_MAP_TYPE_LRU_HASH yang digunakan untuk menghubungkan event sys_enter dan sys_exit.

```
struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, 16384);
    __type(key, __u64);
    __type(value, struct cpuwatch_inflight_syscall);
} inflight SEC(".maps");
```

Di mana `cpuwatch_inflight_syscall` menyimpan data yang meliputi timestamp awal, nomor syscall, dan argumen syscall.

```
// event stats
struct cpuwatch_inflight_syscall {
    __u64 timestamp_ns;
    __s64 syscall_nr;
    __u64 args[6];
};
```

Pada `sys_enter`, data disimpan:

```
bpf_map_update_elem(&target_inflight, &pid_tgid, &syscall_nr, BPF_ANY);
```

Pada `sys_exit`, state dibaca kembali untuk menghitung durasi syscall:

```
state = bpf_map_lookup_elem(&inflight, &pid_tgid);
```

Setelah selesai, entry dihapus dari map.

e. State Rate Limiting

Map `target_inflight` menggunakan `BPF_MAP_TYPE_LRU_HASH`, digunakan sebagai penanda sementara bahwa suatu proses sedang menjalankan syscall yang menjadi target rate limiting.

```
struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, 4096);
    __type(key, __u64);
    __type(value, __s64);
} target_inflight SEC(".maps");
```

```

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_HASH);
    __uint(max_entries, 1024);
    __type(key, struct cpuwatch_rate_key);
    __type(value, struct cpuwatch_rate_state);
} rate_states SEC(".maps");

```

Di mana state utama rate limiter disimpan pada `rate_states`. Selain itu, rate limiter menyimpan key `cpuwatch_rate_key` dan value `cpuwatch_rate_state` dengan nilai sebagai berikut

```

struct cpuwatch_rate_key {
    __u32 tgid;
};

struct cpuwatch_rate_state {
    __u64 window_start_ns;
    __u32 count;
    __u32 denied;
};

```


Hook Point yang Dipilih dan Alasannya

Berikut pemilihan hook yang dipilih beserta alasannya

Hook	Alasan
raw_syscalls/sys_enter	Digunakan sebagai titik observasi utama untuk seluruh syscall yang masuk. Raw syscall tracepoint menyediakan nomor syscall dan argumen mentah secara langsung serta lebih stabil untuk observasi dibanding bergantung pada fungsi internal kernel tertentu.
raw_syscalls/sys_exit	Melengkapi data sys_enter dengan hasil eksekusi syscall. Timestamp entry yang disimpan pada map inflight digunakan untuk menghitung durasi syscall, sedangkan argumen entry digunakan kembali agar event exit tetap memiliki konteks syscall secara lengkap.
sched/sched_switch	Digunakan untuk mengamati aktivitas scheduler secara langsung. Hook ini juga menjadi sumber counter context switch pada setiap CPU untuk statistik live.
kprobe/handle_mm_fault	(implementasi bonus) Digunakan untuk fitur page fault tracing karena handle_mm_fault merupakan bagian dari jalur penanganan page fault pada kernel dan menyediakan alamat memori yang memicu fault.
fentry/fexit/do_sys_openat2	(implementasi bonus) Digunakan untuk membaca argumen fungsi kernel secara lebih terstruktur dengan informasi tipe dari BTF. fentry/fexit menggunakan BPF trampoline sehingga memiliki overhead rendah dan tidak bergantung pada mekanisme breakpoint seperti kprobe. .
kprobe/kretprobe/do_sys_openat2	(implementasi bonus) Digunakan sebagai pendekatan pembandingan terhadap fentry/fexit. Kprobe dapat dipasang berdasarkan simbol fungsi kernel, tetapi lebih bergantung pada keberadaan dan perubahan implementasi internal kernel.
lsm/file_open	(implementasi bonus) Digunakan sebagai enforcement point untuk fitur rate limiting. Hook BPF LSM dapat mengembalikan error sehingga operasi open yang telah melewati threshold dapat dihentikan dengan -EAGAIN sebelum operasi berhasil diselesaikan.

Implementasi Bonus

a. Page Fault Hook

Implementasi hook kprobe/handle_mm_fault diimplementasikan dalam

```
SEC("kprobe/handle_mm_fault")
int BPF_KPROBE(handle_page_fault, struct vm_area_struct *vma,
               unsigned long address, unsigned int flags)
{
```

Di mana hook tersebut dapat diaktifkan dengan --pagefault. Di mana saat pagefault terjadi, program akan mengambil TGID proses dan memeriksa filternya. Jika semua sudah aman, counter statistik berupa atribut pagefault akan diupdate agar sesuai.

```
__u32 tgid = bpf_get_current_pid_tgid() >> 32;

if (!cfg || !cfg->pagefault_enabled || !process_selected(cfg, tgid))
    return 0;

cpu_stats = get_stats();
if (cpu_stats)
    increment_stat(&cpu_stats->page_faults);
```

Untuk mengetes pagefault bisa dilakukan dengan contoh command berikut

```
sudo ./build/cpuwatch \
--events \
--pagefault \
--pid <PID>
```

b. CPU-Level Rate Limiting

Implementasi bonus ini menggunakan fungsi bpf_ktime_get_ns() dan map per core untuk membatasi syscall openat per process per detik, dimana ketika melebihi threshold, syscall akan direject. Cara tersebut terjadi dengan awalnya program akan menandai syscall sys_enter (menjadi semacam tempat untuk melihat mana syscall-syscall yang masuk) yang dimiliki suatu target cpu dan nantinya yang ditandai tersebut akan dihook oleh lsm/file_open.

\ada hook lsm/file_open ini akan membuat suatu state rate limiter dan mulai menghitung waktu menggunakan bpf_ktime_get_ns() hingga suatu window, jika syscall melebihi threshold, maka state->denied counter akan diupdate. Proses penlokasian syscallnya sendiri digunakan dengan mekanisme -EAGAIN

```

SEC("lsm/file_open")
int BPF_PROG(enforce_file_open, struct file *file, int ret)
{
    ...
    now = bpf_ktime_get_ns();
    state = bpf_map_lookup_elem(&rate_states, &key);
    if (!state) {
        initial.window_start_ns = now;
        initial.count = 1;
        bpf_map_update_elem(&rate_states, &key, &initial, BPF_NOEXIST);
        state = bpf_map_lookup_elem(&rate_states, &key);
        if (!state)
            return 0;
    } else if (now - state->window_start_ns >= CPUWATCH_NSEC_PER_SEC) {
        state->window_start_ns = now;
        state->count = 1;
        state->denied = 0;
    } else {
        state->count++;
    }

    if (state->count <= cfg->rate_limit_threshold)
        return 0;
    state->denied++;
    cpu_stats = get_stats();
    if (cpu_stats)
        increment_stat(&cpu_stats->rate_limited);
    ...
}

```

Pengujian dapat dilakukan dengan awalnya menandakan suatu PID yang masuk

```

./build/rate_limit_test \
--attempts 200 \
--cpu 0

```

Lalu mencatat PID yang dimasukkan, dan menjalankan

```

sudo ./build/cpuwatch \
--events \
--rate-limit-pid <PID> \
--rate-limit 50

```

c. Pembuktian Tidak Ada Data Race

Implementasi bonus penjaminan tidak adanya data race dilakukan menggunakan utility tesing `race_test`, di mana berfungsi sebagai pendeteksi seluruh CPU yang diizinkan oleh affinity mask, membuat satu thread worker untuk tiap CPU, mem-pin worker ke CPU tersebut, lalu setiap worker menjalankan `getpid` sebanyak N kali. Karena stats menggunakan `PERCPU_ARRAY`, setiap worker menghasilkan update pada slot CPU-nya sendiri seharusnya dapat selalu konsisten. Sebagai bukti konsistensi ini, diperoleh dengan membandingkan delta syscall dari tools `race_test` menggunakan command ini

```
./build/race_test --calls 1000000
```

Hasil seharusnya memberikan expected seperti berikut

```
sudo scripts/race_test.sh 1000000
[sudo] password for VM:
CPU0 expected=1000000 observed=1000000 difference=0
CPU1 expected=1000000 observed=1000000 difference=0
CPU2 expected=1000000 observed=1000000 difference=0
CPU3 expected=1000000 observed=1000000 difference=0
TOTAL expected=4000000 observed=4000000 difference=0
Race validation passed; evidence saved under evidence/.
```

d. Fentry/Fexit

Implementasi ini akan straight forward, di mana kita akan menggunakan `fentry/fexit` untuk membaca argumen syscall tertentu tanpa mengubahnya

```
SEC("fentry/do_sys_openat2")
int BPF_PROG(fentry_do_sys_openat2, int dfd, const char *filename,
             struct open_how *how)
{
    struct cpuwatch_config *cfg = get_config();

    if (!cfg || !cfg->fentry_enabled)
        return 0;
    return emit_open_entry(dfd, filename, how,
CPUWATCH_EVENT_FENTRY_OPEN);
}
```

```

SEC("fexit/do_sys_openat2")
int BPF_PROG(fexit_do_sys_openat2, int dfd, const char *filename, struct
open_how *how, long ret)
{
    struct cpuwatch_config *cfg = get_config();

    if (!cfg || !cfg->fentry_enabled)
        return 0;
    return emit_open_exit(ret, CPUWATCH_EVENT_FEXIT_OPEN);
}

```

Perbedaannya dengan hook kprobe antara lain

Aspek	Fentry/fexit	Kprobe/kretprobe
Informasi argument	Menggunakan tipe dari BTF	Lebih bergantung pada calling convention
Attachment	Trampoline	Probe/breakpoint
Overhead	Umumnya lebih rendah	Umumnya lebih tinggi
Stabilitas	Lebih baik jika BTF tersedia	Lebih sensitif terhadap perubahan internal kernel
Dependency	Membutuhkan target di BTF	Membutuhkan symbol kernel
Tujuan	Observer typed	Observer pembanding

Hasil Verifier dan Benchmark

a. Verifier

Hasil verifier terverifikasi dengan baik, dimana jika kita menjalankan suatu program bpf yang sengaja salah

```
make verifier
cd scripts/
sudo ./verifier_test.sh
Verifier rejected the intentionally invalid object as expected.
Production object loaded successfully. Evidence saved under evidence/
```

Verifier akan menolaknya.

That said, ada kasus di mana saat implementasi terdapat kesalahan sehingga verifier menolaknya, ketika kita melakukan for loop pada pointer ctx

```
#pragma unroll
for (i = 0; i < 6; i++)
    state.args[i] = ctx->args[i];
```

Di mana secara logika valid, tetapi setelah dikompilasi LLVM menghasilkan instruksi pointer reference yang tidak dapat diprediksi (semacam dereference pointernya menurut verifier sudah ambigu jadi tidak tahu apakah sekarang menunjukkan alamat yang masih sah atau tidak). Fixnya adalah kita melakukan semacam explicit pointer dereference

```
state.args[0] = ctx->args[0];
state.args[1] = ctx->args[1];
state.args[2] = ctx->args[2];
state.args[3] = ctx->args[3];
state.args[4] = ctx->args[4];
state.args[5] = ctx->args[5];
```

Sehingga nanti dereference pointer pada compile akan menjadi eksplisit offsetnya (karena manual semua). Hasil ini diterima oleh verifier dan program eBPF kita dapat dimasukkan ke kernel.

b. Benchmark

Benchmark tiap hook yang dipasang memiliki hasil sebagai berikut

Mode	Workload	Rata-rata Waktu	Overhead
------	----------	-----------------	----------

Baseline	1.000.000 getpid() tanpa CPUWatch	1,203 s	0%
Aggregate	Program aktif dengan agregasi per-CPU tanpa event ring buffer	1,682 s	39,80%
Ringbuf	Program aktif dengan agregasi dan pengiriman event melalui ring buffer	2,048 s	70,30%
Full	Program dengan event logging serta fitur tambahan diaktifkan	2,093 s	74,00%

Benchmark ini mengukur overhead program dengan membandingkan waktu eksekusi 1.000.000 pemanggilan getpid() pada beberapa konfigurasi. Mode Aggregate menunjukkan biaya tambahan dari hook dan pembaruan statistik per-CPU, sedangkan Ringbuf menambahkan biaya pengiriman setiap event dari kernel ke user space. Mode Full menjalankan konfigurasi CPUWatch dengan lebih banyak fitur aktif. Hasil menunjukkan bahwa agregasi per-CPU memiliki overhead paling rendah di antara mode monitoring, sedangkan pengiriman event melalui ring buffer memberikan tambahan overhead yang cukup besar.

Selain itu, terdapat juga benchmark yang membandingkan Kprobe dan Fentry

Mode	Workload	Rata-rata Waktu	Overhead
Open Baseline	1.000.000 openat() tanpa CPUWatch	6,154 s	0%
Kprobe	openat() dengan kprobe/kretprobe	9,608 s	56,13%
Fentry	openat() dengan fentry/fexit	9,222 s	49,85%

Pengukuran kedua membandingkan overhead fentry/fexit dan kprobe/kretprobe pada 1.000.000 pemanggilan openat(). Open Baseline digunakan sebagai acuan karena workload yang diuji berbeda dari benchmark getpid(). Pada pengujian ini, fentry/fexit menghasilkan waktu rata-rata 9,222 detik dengan overhead 49,85%, sedangkan kprobe/kretprobe menghasilkan 9,608 detik dengan overhead 56,13%. Hasil tersebut menunjukkan bahwa pada environment pengujian ini, fentry/fexit memiliki overhead yang lebih rendah dibandingkan kprobe/kretprobe.